

Contrôle de qualité

Debugger, profiler et durcir la qualité du code

Module 6/7

Objectifs du module

Durée : 3h30

- Debugger avec le debugger V8 et des outils externes
- Profiler une application Node.js (CPU et mémoire)
- Améliorer la qualité du code et la gestion des erreurs dans Node.js

Plan du module

1. Debugger V8 et outils externes

- `node --inspect` / `--inspect-brk`, Chrome DevTools, VS Code, breakpoints, watch

2. Profiling

- CPU profile, `--prof` / `--prof-process`, clinic.js (doctor/flame), heap snapshots

3. Qualité du code et gestion des erreurs

- ESLint, conventions, complexité ; erreurs opérationnelles vs programmeur, middleware d'erreur, sortie propre

1. Debugger V8 et outils externes

Le debugger intégré V8

Concept : Node.js embarque le moteur V8 qui expose le **Inspector Protocol**. On l'active au lancement pour piloter l'exécution depuis un client (Chrome DevTools, VS Code).

`--inspect`

Ouvre le port d'inspection (9229) et laisse l'application démarrer immédiatement.

`--inspect-brk`

Ouvre le port et **suspend** l'exécution à la première ligne, le temps d'attacher un client.

```
node --inspect server.js
node --inspect-brk server.js
node --inspect=0.0.0.0:9229 server.js    # exposer sur une autre interface
```

Attacher Chrome DevTools

Concept : le client d'inspection se connecte au port WebSocket exposé par V8 et offre breakpoints, watch, console et profiler.

- Lancer `node --inspect-brk server.js`
- Ouvrir `chrome://inspect` dans Chrome / Edge
- Cliquer sur **inspect** sous la cible « Remote Target »
- L'onglet **Sources** permet de poser des breakpoints, **Console** d'évaluer

```
# La sortie console affiche l'URL devtools à coller dans le navigateur
Debugger listening on ws://127.0.0.1:9229/abcd-1234
For help, see: https://nodejs.org/en/docs/inspector
```

Concept : VS Code intègre un client Inspector. On décrit la configuration dans

`.vscode/launch.json`.

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Debug API",
      "program": "${workspaceFolder}/server.js",
      "skipFiles": ["<node_internals>/**"],
      "console": "integratedTerminal"
    }
  ]
}
```

Mode **attach** pour un process déjà lancé avec `--inspect` :

```
{ "type": "node", "request": "attach", "name": "Attach", "port": 9229 }
```

Breakpoints et watch

Concept : maîtriser le pas-à-pas permet d'inspecter l'état réel sans multiplier les `console.log`.

Breakpoints

- Clic dans la gouttière (ligne)
- Conditionnel : `req.id === '42'`
- `debugger;` directement dans le code
- Logpoint : trace sans arrêt

Contrôle d'exécution

- Continue (F5)
- Step Over (F10)
- Step Into (F11)
- Step Out (Shift+F11)

Watch : ajouter des expressions (`req.body` , `user?.role`) évaluées à chaque arrêt.
La vue **Scope** montre variables locales et closures.

Debugger une route Express

```
import express from "express";
const app = express();

app.get("/users/:id", (req, res) => {
  const id = Number(req.params.id);
  debugger; // arrêt ici en mode --inspect
  const user = findUser(id); // Step Into pour entrer dans findUser
  res.json(user);
});

app.listen(3000);
```

Lancer `node --inspect-brk server.js`, poser un watch sur `id` et `user`, appeler `GET /users/5`.

Démonstration

Voir `demos/demo-06.md`

Debugger pas à pas avec `node --inspect-brk` (breakpoint dans une route Express), profiler le CPU sur une route lente, et gestion d'erreurs centralisée.

2. Profiling

Pourquoi profiler

Concept : le profiling mesure **où** le temps CPU et la mémoire sont consommés, au lieu de deviner. Indispensable avant toute optimisation.

- Profiling **CPU** : quelles fonctions monopolisent le thread d'événements
- Profiling **mémoire** : fuites, objets retenus, croissance du heap
- Règle : mesurer → identifier le goulot → optimiser → re-mesurer

```
# Un profil pris en production reflète la charge réelle,  
# pas un microbenchmark isolé.
```

CPU profile via DevTools

Concept : l'onglet **Profiler** / **Performance** de Chrome DevTools enregistre un profil CPU échantillonné.

- Attacher DevTools (`--inspect`)
- Onglet **Profiler** > Record
- Générer la charge (appeler la route lente)
- Stop > analyser le **flame chart** et le tri **Self time / Total time**

```
node --inspect server.js  
# autocannon pour générer de la charge pendant l'enregistrement  
npx autocannon -d 10 http://localhost:3000/slow
```

`--prof` et `--prof-process`

Concept : V8 peut écrire un log de profiling brut, ensuite agrégé en rapport lisible.

```
# 1. Génère un fichier isolate-*.log
node --prof server.js
# (générer la charge puis arrêter le process)

# 2. Transforme le log en rapport texte
node --prof-process isolate-0x*.log > processed.txt
```

Le rapport `processed.txt` liste les **ticks** par fonction, regroupés par catégorie (JavaScript, C++, GC). La section **Summary** indique la répartition du temps.

clinic.js : doctor

Concept : `clinic doctor` diagnostique automatiquement la santé du process (event loop, CPU, mémoire, I/O actifs) et oriente vers la cause.

```
npm i -g clinic          # ou npx clinic
clinic doctor -- node server.js
# Générer la charge, puis Ctrl+C
npx autocannon http://localhost:3000/slow
```

À l'arrêt, clinic ouvre un rapport HTML avec des recommandations : « event loop blocking », « I/O », etc.

clinic.js : flame

Concept : `clinic flame` produit un **flamegraph** des fonctions JavaScript par temps CPU cumulé. La largeur d'une barre = temps passé.

```
clinic flame -- node server.js
npx autocannon -d 10 http://localhost:3000/slow
# Ctrl+C : ouverture du flamegraph interactif
```

- Barres larges en haut = fonctions « feuilles » coûteuses
- Cliquer pour zoomer sur une pile
- Cibler la fonction la plus large avant d'optimiser

Heap snapshots

Concept : un **heap snapshot** capture tous les objets vivants du tas V8 à un instant. La comparaison de deux snapshots révèle les fuites.

```
import v8 from "node:v8";
import fs from "node:fs";

// Écrit un snapshot programmatique
const file = v8.writeHeapSnapshot();
console.log("Snapshot écrit :", file);
```

Dans DevTools, onglet **Memory** > **Heap snapshot** > comparer (Comparison).
Chercher les **Delta** positifs et les **Detached** nodes.

3. Qualité du code et gestion des erreurs

ESLint : linter et conventions

Concept : ESLint analyse statiquement le code pour appliquer des règles de style, détecter des bugs probables et limiter la complexité.

```
npm i -D eslint
npx eslint --init          # génère eslint.config.js (flat config)
npx eslint .
```

```
// eslint.config.js (flat config, ESLint 9)
import js from "@eslint/js";
export default [
  js.configs.recommended,
  { rules: { "no-unused-vars": "error", complexity: ["warn", 10] } }
];
```

Complexité et conventions

Concept : la complexité cyclomatique mesure le nombre de chemins d'exécution. Trop élevée = code difficile à tester et à maintenir.

Règles utiles

- `complexity` (cyclomatique)
- `max-depth`, `max-lines-per-function`
- `no-shadow`, `eqeqeq`
- `no-floating-promises` (TS)

Conventions

- nommage cohérent
- une responsabilité par fonction
- early return pour réduire l'imbrication
- Prettier pour le formatage

```
npm i -D prettier eslint-config-prettier # désactive les règles de style en conflit
```

Erreurs : opérationnelles vs programmeur

Concept : distinguer ces deux familles change la stratégie de réaction.

Opérationnelles

Conditions attendues à l'exécution : 404, validation invalide, timeout réseau, DB indisponible.

→ **Gérer** : répondre proprement, réessayer.

Programmeur (bugs)

`undefined is not a function`, mauvais usage d'API, invariants cassés.

→ **Ne pas masquer** : corriger le code ; en dernier recours, crasher proprement.

Classes d'erreurs personnalisées

Concept : des classes typées portent un statut HTTP et un flag `isOperational`, exploitables par le middleware.

```
export class AppError extends Error {
  constructor(message, statusCode = 500, isOperational = true) {
    super(message);
    this.name = this.constructor.name;
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    Error.captureStackTrace(this, this.constructor);
  }
}

export class NotFoundError extends AppError {
  constructor(msg = "Ressource introuvable") { super(msg, 404); }
}

export class ValidationError extends AppError {
  constructor(msg = "Données invalides") { super(msg, 400); }
}
```

Middleware d'erreur Express

Concept : un middleware à 4 arguments (err, req, res, next) centralise la réponse d'erreur. Express 5 propage automatiquement les rejets de handlers async.

```
// Doit être enregistré APRÈS toutes les routes
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  if (!err.isOperational) {
    console.error("Erreur programmeur :", err);    // log complet
  }
  res.status(status).json({
    error: err.isOperational ? err.message : "Erreur interne"
  });
});
```

En Express 4, encapsuler les handlers async (asyncHandler) pour transmettre les rejets à next .

Handlers de process et sortie propre

Concept : capturer les erreurs non gérées au niveau du process pour logger puis sortir proprement — un état corrompu ne doit pas perdurer.

```
process.on("uncaughtException", (err) => {  
  console.error("uncaughtException", err);  
  process.exit(1); // sortir, laisser l'orchestrateur relancer  
});  
  
process.on("unhandledRejection", (reason) => {  
  console.error("unhandledRejection", reason);  
  process.exit(1);  
});  
  
process.on("SIGTERM", () => {  
  server.close(() => process.exit(0)); // graceful shutdown  
});
```


Travaux pratiques

Fichier : `tp/tp-06.md`

Outiller une API Express : debug (`--inspect` + VS Code), profiling (identifier une route lente), lint (ESLint) et gestion d'erreurs robuste (classes + middleware + handlers de process).

Récapitulatif

- `--inspect` / `--inspect-brk` ouvrent le V8 Inspector ; DevTools et VS Code attachent breakpoints et watch
- Profiler avant d'optimiser : DevTools, `--prof` / `--prof-process` , clinic doctor/flame, heap snapshots
- ESLint applique conventions et limite la complexité
- Séparer erreurs opérationnelles et programmeur, centraliser via middleware, gérer `uncaughtException` / `unhandledRejection` et la sortie propre

Cheatsheet : `cheatsheet/module-06.md`

Questions ?

Module suivant : Modules avancés
